

TOTAL MODULES	CURRICULUM PART	CORE INNOVATIONS	REFERENCE STANDARD
57 Modules + 8 Practice Labs	Part 30 — Modern ECMAScript Standards	Null Safety, Immutability, Private Fields	TC39 Living Standard ES2020–ES2026

🌟 Modern ECMAScript (ES2020+) — আধুনিক জাভাস্ক্রিপ্ট সিনট্যাক্স, ইমিউটেবিলিটি ও পারফরম্যান্স

জাভাস্ক্রিপ্ট একটি জীবন্ত ভাষা যা প্রতি বছর নতুন নতুন শক্তিশালী ফিচার ও সিনট্যাক্টিক সুবিধায় সমৃদ্ধ হচ্ছে। কোডকে নাল-পয়েন্টার এরর থেকে রক্ষা করা, স্টেট মিউটেশনের ঝুঁকিহীন ইমিউটেবল অ্যারে মেথড এবং ক্লাস এনক্যাপসুলেশন নিশ্চিত করাই আধুনিক ECMAScript-এর লক্ষ্য। এই চ্যাপ্টারে আমরা **Optional Chaining (?.)**, **Nullish Coalescing (??)**, **Logical Assignments**, **Private Class Fields (#)**, **Top-Level await**, **.at()**, **Immutable Arrays (toSorted, toSpliced, with)**, **Object.groupBy**, **Promise.withResolvers** এবং Temporal API শিখব।

- 🌟 1. Optional Chaining

🌟 2. Nullish Coalescing

🌟 3. Logical Assignments

🌟 4. Private Class Fields

🌟 5. Top-Level await

🌟 6. Negative Index (.at)

🌟 7. Immutable Arrays

🌟 8. Object.groupBy()

🌟 9. Promise.withResolvers

🌟 10. Modern React Toolkit

Part 01 — ECMAScript Standards, Specification & Evolution (36.1 – 36.3)

36.1 ECMAScript কী?

ECMAScript হলো JavaScript language-এর standard/specification। সহজভাবে:

🌟 FULL-STACK MODERN JAVASCRIPT TOOLKIT

ECMAScript

↓

JavaScript language-এর specification

↓

Browser / Node.js

↓

JavaScript চালায়

তাই:

🌟 MODERN JAVASCRIPT ARCHITECTURE FLOW

JavaScript ≠ ECMAScript

আরও সহজভাবে:

💡 ECMAScript হলো rulebook, আর JavaScript সেই specification-এর একটি বাস্তব implementation।

36.2 ES Version কী?

JavaScript সময়ের সাথে নতুন feature পেয়েছে। যেমন:

🌟 MODERN ECMASCRIPT TIMELINE & EVOLUTION

ES2015 → let, const, arrow function, class
ES2016 → Array.includes()
ES2017 → async/await
ES2018 → object rest/spread
ES2019 → flat(), flatMap()
ES2020 → ??, ?., BigInt, Promise.allSettled()
ES2021 → ??=, ||=, &&=, replaceAll()
ES2022 → class fields, #private, top-level await
ES2023 → findLast(), toSorted(), toReversed(), toSpliced()
ES2024 → Promise.withResolvers(), Object.groupBy(), Map.groupBy()
ES2025 → নতুন standardized features

36.3 কেন Modern JavaScript Features শেখা দরকার?

পুরোনো style:

JAVASCRIPT SOURCE

JAVASCRIPT

```
if (user && user.address && user.address.city) {  
  console.log(user.address.city);  
}
```

Modern JavaScript:

JAVASCRIPT SOURCE

JAVASCRIPT

```
console.log(user?.address?.city);
```

পুরোনো style:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const name = user.name || "Guest";
```

Modern:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const name = user.name ?? "Guest";
```

Modern syntax:

- code ছোট করে
- readability বাড়ায়
- bugs কমাতে সাহায্য করে
- React/Node code cleaner করে
- modern libraries বুঝতে সাহায্য করে

36.4 Optional Chaining ?.

এটি ES2020-এর গুরুত্বপূর্ণ feature। ধরো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const user = {
  name: "Ripon"
};
```

তুমি লিখলে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
console.log(user.address.city);
```

তাহলে error হবে:

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

Cannot read properties of undefined

কারণ:

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

user.address → undefined

✦ Optional Chaining

JAVASCRIPT SOURCE

JAVASCRIPT

```
console.log(user.address?.city);
```

Output:

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

undefined

আরও:

JAVASCRIPT SOURCE

JAVASCRIPT

```
console.log(user?.address?.city);
```

এখন property missing হলেও error হবে না।

36.5 Optional Chaining with Functions

ধরো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const user = {};
```

আগে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
if (user.login) {
  user.login();
}
```

Modern:

JAVASCRIPT SOURCE

JAVASCRIPT

```
user.login?.();
```

যদি function থাকে → call হবে। না থাকলে → error না দিয়ে undefined return করবে।

36.6 Optional Chaining with Arrays

JAVASCRIPT SOURCE

JAVASCRIPT

```
const users = [];  
  
console.log(users[0]?.name);
```

Output:

✨ MODERN JAVASCRIPT ARCHITECTURE FLOW

undefined

36.7 Nullish Coalescing ??

?? ব্যবহার করা হয় যখন value শুধু null অথবা undefined কিনা check করতে চাই।

JAVASCRIPT SOURCE

JAVASCRIPT

```
const username = null ?? "Guest";  
  
console.log(username);
```

Output:

✨ MODERN JAVASCRIPT ARCHITECTURE FLOW

Guest

36.8 || vs ??

এটা খুব গুরুত্বপূর্ণ।

JAVASCRIPT SOURCE

JAVASCRIPT

```
const value = 0 || 100;

console.log(value);
```

Output:

✨ MODERN JAVASCRIPT ARCHITECTURE FLOW

100

কারণ 0 হলো falsy। কিন্তু:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const value = 0 ?? 100;

console.log(value);
```

Output:

✨ MODERN JAVASCRIPT ARCHITECTURE FLOW

0

কারণ ?? শুধু:

✨ MODERN JAVASCRIPT ARCHITECTURE FLOW

null
undefined

কে missing হিসেবে ধরে।

✨ Comparison

Value	`value`	100`	value ?? 100
null	100	100	
undefined	100	100	
0	100	0	
false	100	false	
""	100	""	

36.9 Logical Assignment Operators

ES2021-এ এসেছে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
&&=  
||=  
??=
```

✨ ||=

JAVASCRIPT SOURCE

JAVASCRIPT

```
let username;  
  
username ||= "Guest";  
  
console.log(username);
```

Output:

✨ MODERN JAVASCRIPT ARCHITECTURE FLOW

Guest

Equivalent concept:

JAVASCRIPT SOURCE

JAVASCRIPT

```
username = username || "Guest";
```

36.10 ??=

JAVASCRIPT SOURCE

JAVASCRIPT

```
let username = null;  
  
username ??= "Guest";  
  
console.log(username);
```

Output:

✨ MODERN JAVASCRIPT ARCHITECTURE FLOW

Guest

কিন্তু:

JAVASCRIPT SOURCE

JAVASCRIPT

```
let score = 0;  
  
score ??= 100;  
  
console.log(score);
```

Output:

✨ MODERN JAVASCRIPT ARCHITECTURE FLOW

0

36.11 &&=

JAVASCRIPT SOURCE

JAVASCRIPT

```
let user = {  
  loggedIn: true  
};  
  
user.loggedIn &&= false;  
  
console.log(user.loggedIn);
```

Output:

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

false

এটি conceptually:

JAVASCRIPT SOURCE

JAVASCRIPT

```
user.loggedIn = user.loggedIn && false;
```

36.12 String.prototype.replaceAll()

আগে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
"apple apple apple".replace(/apple/g, "orange");
```

Modern:

JAVASCRIPT SOURCE

JAVASCRIPT

```
"apple apple apple".replaceAll(  
  "apple",  
  "orange"  
);
```

Output:

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

orange orange orange

36.13 Promise.allSettled()

আগে:

JAVASCRIPT SOURCE

JAVASCRIPT

`Promise.all()`

একটি Promise reject করলে পুরো Promise.all() reject করে। কিন্তু:

JAVASCRIPT SOURCE

JAVASCRIPT

`Promise.allSettled()`

সব Promise-এর result জানায়। Example:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const promises = [
  Promise.resolve("Success"),
  Promise.reject("Failed"),
  Promise.resolve("Success 2")
];

Promise.allSettled(promises)
  .then(results => console.log(results));
```

Result conceptually:

JAVASCRIPT SOURCE

JAVASCRIPT

```
[
  { status: "fulfilled", value: "Success" },
  { status: "rejected", reason: "Failed" },
  { status: "fulfilled", value: "Success 2" }
]
```

♦ Real-Life Example

তুমি ৩টি API call করলে:

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

API 1 → Success
 API 2 → Failed
 API 3 → Success

তবুও তুমি তিনটির result-ই জানতে চাও। তখন:

JAVASCRIPT SOURCE

JAVASCRIPT

`Promise.allSettled()`

খুব usefull

36.14 Promise.any()

ES2021-এর আরেকটি গুরুত্বপূর্ণ feature।

JAVASCRIPT SOURCE

JAVASCRIPT

```
Promise.any([
  Promise.reject("Server 1 failed"),
  Promise.resolve("Server 2 success"),
  Promise.resolve("Server 3 success")
])
  .then(result => console.log(result));
```

Output:

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

Server 2 success

কারণ Promise.any() প্রথম fulfilled Promise-এর result দেয়।

36.15 AggregateError

যদি:

JAVASCRIPT SOURCE

JAVASCRIPT

```
Promise.any([
  Promise.reject("A"),
  Promise.reject("B"),
  Promise.reject("C")
]);
```

সব Promise reject করলে `Promise.any()` একটি `AggregateError` reject করে। এতে multiple errors পাওয়া যায়।

Part 05 — Code Readability: Numeric Separators & Logical Assignment Patterns (36.16 – 36.17)

36.16 Numeric Separators

বড় number পড়া সহজ করার জন্য:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const population = 170_000_000;
```

এটি equivalent:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const population = 170000000;
```

আর:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const price = 1_299_999;
```

এটা শুধু readability-এর জন্য।

36.17 Logical Assignment বাস্তব Example

ধরো configuration:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const config = {
  theme: null,
  debug: false
};
```

Default theme:

JAVASCRIPT SOURCE

JAVASCRIPT

```
config.theme ??= "light";
```

Output:

JAVASCRIPT SOURCE

JAVASCRIPT

```
console.log(config.theme);
```

★ MODERN JAVASCRIPT ARCHITECTURE FLOW

```
light
```

36.18 Private Class Fields #

ES2022-এ JavaScript class-এ true private fields এসেছে।

JAVASCRIPT SOURCE

JAVASCRIPT

```
class BankAccount {  
  
  #balance = 0;  
  
  deposit(amount) {  
    this.#balance += amount;  
  }  
  
  getBalance() {  
    return this.#balance;  
  }  
}
```

Use:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const account = new BankAccount();  
  
account.deposit(100);  
  
console.log(account.getBalance());
```

Output:

🌟 MODERN JAVASCRIPT ARCHITECTURE FLOW

100

কিন্তু:

JAVASCRIPT SOURCE

JAVASCRIPT

```
console.log(account.#balance);
```

এটা বাইরে থেকে access করা যাবে না।

36.19 Public Class Fields

আগে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
class User {  
  
  constructor(name) {  
    this.name = name;  
  }  
  
}
```

Modern:

JAVASCRIPT SOURCE

JAVASCRIPT

```
class User {  
  
  name = "Guest";  
  
  age = 0;  
  
}
```

তরপর:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const user = new User();  
  
console.log(user.name);
```

Output:

✨ MODERN JAVASCRIPT ARCHITECTURE FLOW

Guest

36.20 Static Class Fields

JAVASCRIPT SOURCE

JAVASCRIPT

```
class Config {  
  
  static appName = "My App";  
  
}
```

Access:

JAVASCRIPT SOURCE

JAVASCRIPT

```
console.log(Config.appName);
```

Output:

✨ MODERN JAVASCRIPT ARCHITECTURE FLOW

My App

Instance দিয়ে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const config = new Config();  
  
console.log(config.appName);
```

এখানে undefined হবে কারণ appName static।

36.21 Static Methods

JAVASCRIPT SOURCE

JAVASCRIPT

```
class MathUtil {  
  
  static square(n) {  
    return n * n;  
  }  
  
}
```

Call:

JAVASCRIPT SOURCE

JAVASCRIPT

```
console.log(MathUtil.square(5));
```

Output:

🌟 MODERN JAVASCRIPT ARCHITECTURE FLOW

25

36.22 Static Initialization Block

Modern JavaScript-এ:

JAVASCRIPT SOURCE

JAVASCRIPT

```
class App {  
  
  static version;  
  
  static {  
    App.version = "1.0.0";  
  }  
  
}
```

তারপর:

JAVASCRIPT SOURCE

JAVASCRIPT

```
console.log(App.version);
```

Output:

🌟 MODERN JAVASCRIPT ARCHITECTURE FLOW

1.0.0

Static block class initialization-এর সময় একবার execute হয়।

36.23 Top-Level await

ES2022-এর খুব গুরুত্বপূর্ণ feature। Module-এর top level-এ await ব্যবহার করা যায়। ধরো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const response = await fetch(
  "https://example.com/data"
);

const data = await response.json();

console.log(data);
```

আগে await সাধারণত:

JAVASCRIPT SOURCE

JAVASCRIPT

```
async function load() {
  ...
}
```

এর ভিতরে লাগত। ES Modules-এ top-level await ব্যবহার করা যায়।

36.24 Top-Level Await কোথায় Useful?

বিশেষ করে:

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

Module initialization
Configuration loading
Database setup
Dynamic resource loading

তবে module execution dependency-এর কারণে ব্যবহার করার সময় performance ও startup behavior বিবেচনা করতে হবে।

36.25 .at()

ES2022-এ at() এসেছে। আগে array-এর last element:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const numbers = [10, 20, 30];

console.log(numbers[numbers.length - 1]);
```

Modern:

JAVASCRIPT SOURCE

JAVASCRIPT

```
console.log(numbers.at(-1));
```

Output:

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

30

36.26 Negative Index

JAVASCRIPT SOURCE

JAVASCRIPT

```
const numbers = [10, 20, 30, 40];

console.log(numbers.at(-1));
console.log(numbers.at(-2));
```

Output:

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

```
40
30
```

String-এর ক্ষেত্রে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
console.log("Hello".at(-1));
```

Output:

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

```
o
```

36.27 Object.hasOwn()

আগে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
Object.prototype.hasOwnProperty.call(
  user,
  "name"
);
```

Modern:

JAVASCRIPT SOURCE

JAVASCRIPT

```
Object.hasOwn(user, "name");
```

Example:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const user = {
  name: "Ripon"
};

console.log(
  Object.hasOwn(user, "name")
);
```

Output:

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

```
true
```

এটি property **নিজের property কিনা** check করে।

36.28 Error.cause

একটি error-এর original cause attach করা যায়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
try {  
  
    throw new Error("Database failed");  
  
} catch (error) {  
  
    throw new Error(  
        "Could not load user",  
        { cause: error }  
    );  
  
}
```

পরে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
catch (error) {  
    console.log(error.message);  
    console.log(error.cause);  
}
```

এটি debugging এবং error chaining-এর জন্য usefull

Part 08 — Array Querying & Immutable Methods (toSorted, toSpliced, with) (36.29 – 36.36)

36.29 Array.findLast()

ES2023-এর feature। ধরো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const numbers = [10, 20, 30, 20];
```

শেষ থেকে খুঁজতে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const result = numbers.findLast(  
    n => n === 20  
);  
  
console.log(result);
```

Output:

MODERN JAVASCRIPT ARCHITECTURE FLOW

20

36.30 findLastIndex()

JAVASCRIPT SOURCE

JAVASCRIPT

```
const numbers = [10, 20, 30, 20];  
  
console.log(  
    numbers.findLastIndex(n => n === 20)  
);
```

Output:

MODERN JAVASCRIPT ARCHITECTURE FLOW

3

36.31 Immutable Array Methods

Modern JavaScript-এ একটি গুরুত্বপূর্ণ ধারণা হলো **non-mutating array methods**। ES2023-এ:

JAVASCRIPT SOURCE

JAVASCRIPT

```
toSorted()  
toReversed()  
toSpliced()
```

এসেছে।

36.32 toSorted()

পুরোনো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const numbers = [3, 1, 2];  
  
numbers.sort();  
  
console.log(numbers);
```

sort() original array পরিবর্তন করে। Modern:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const numbers = [3, 1, 2];  
  
const sorted = numbers.toSorted();  
  
console.log(sorted);  
console.log(numbers);
```

Result:

🌟 MODERN JAVASCRIPT ARCHITECTURE FLOW

```
[1, 2, 3]  
[3, 1, 2]
```

Original array unchanged।

36.33 toReversed()

JAVASCRIPT SOURCE

JAVASCRIPT

```
const numbers = [1, 2, 3];  
  
const reversed = numbers.toReversed();  
  
console.log(reversed);  
console.log(numbers);
```

Output:

🌟 MODERN JAVASCRIPT ARCHITECTURE FLOW

```
[3, 2, 1]  
[1, 2, 3]
```

36.34 toSpliced()

splice() original array modify করে। toSpliced() নতুন array return করে।

JAVASCRIPT SOURCE

JAVASCRIPT

```
const numbers = [1, 2, 3, 4];

const result = numbers.toSpliced(1, 2);

console.log(result);
console.log(numbers);
```

Output:

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

```
[1, 4]
[1, 2, 3, 4]
```

36.35 with()

ES2023-এর আরেকটি useful immutable array method। ধরো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const numbers = [10, 20, 30];
```

আগে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
numbers[1] = 100;
```

Original array পরিবর্তিত হবে। Modern:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const updated = numbers.with(1, 100);

console.log(updated);
console.log(numbers);
```

Output:

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

```
[10, 100, 30]
[10, 20, 30]
```

React-এর মতো state-based development-এ immutable style খুব usefull

36.36 Array.prototype.toSorted() কেন React-এ Useful?

ধরো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const [users, setUsers] = useState([ ... ]);
```

খারাপ approach:

JAVASCRIPT SOURCE

JAVASCRIPT

```
users.sort();
setUsers(users);
```

কারণ original array mutate হয়েছে। Better:

JAVASCRIPT SOURCE

JAVASCRIPT

```
setUsers(users.toSorted());
```

অথবা comparator সহ:

JAVASCRIPT SOURCE

JAVASCRIPT

```
setUsers(
  users.toSorted(
    (a, b) => a.name.localeCompare(b.name)
  )
);
```

Part 09 — Data Grouping (Object.groupBy, Map.groupBy) & Unicode (36.37 – 36.41)

36.37 Object.groupBy()

ES2024-এর একটি গুরুত্বপূর্ণ feature! ধরো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const users = [
  { name: "A", role: "admin" },
  { name: "B", role: "user" },
  { name: "C", role: "admin" }
];
```

Group:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const grouped = Object.groupBy(
  users,
  user => user.role
);

console.log(grouped);
```

Conceptually:

JAVASCRIPT SOURCE

JAVASCRIPT

```
{
  admin: [
    { name: "A", role: "admin" },
    { name: "C", role: "admin" }
  ],
  user: [
    { name: "B", role: "user" }
  ]
}
```

36.38 Map.groupBy()

যদি grouping result হিসেবে Map দরকার হয়:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const grouped = Map.groupBy(  
  users,  
  user => user.role  
);
```

Difference:

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

```
Object.groupBy()  
→ Object
```

```
Map.groupBy()  
→ Map
```

36.39 Promise.withResolvers()

ES2024-এর useful Promise feature। আগে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
let resolvePromise;  
  
const promise = new Promise(resolve => {  
  resolvePromise = resolve;  
});
```

Modern:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const {  
  promise,  
  resolve,  
  reject  
} = Promise.withResolvers();
```

তারপর:

JAVASCRIPT SOURCE

JAVASCRIPT

```
resolve("Success");
```

Promise:

JAVASCRIPT SOURCE

JAVASCRIPT

```
promise.then(console.log);
```

Output:

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

```
Success
```

এটি বিশেষ করে custom async abstractions, event-based systems এবং Promise control-এর ক্ষেত্রে useful।

36.40 String.isWellFormed()

Unicode string-এর malformed surrogate আছে কিনা check করতে ব্যবহার করা যায়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
const text = "Hello";

console.log(text.isWellFormed());
```

Output:

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

true

36.41 toWellFormed()

Malformed Unicode থাকলে replacement character ব্যবহার করে well-formed string তৈরি করতে পারে।

JAVASCRIPT SOURCE

JAVASCRIPT

```
const text = "Hello";

const safeText = text.toWellFormed();

console.log(safeText);
```

Output:

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

Hello

Part 10 — Emerging Standards: Temporal API & Explicit Resource Management (36.42 – 36.43)

36.42 Temporal সম্পর্কে গুরুত্বপূর্ণ কথা

অনেক JavaScript developer Temporal নিয়ে শুনছেন। Temporal-এর উদ্দেশ্য:

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

Date-এর limitations কমানো

যেমন:

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

Date
Time
Timezone
Duration
Calendar

তবে গুরুত্বপূর্ণ বিষয়:

💡 Temporal দীর্ঘ সময় proposal পর্যায়ে ছিল এবং standardization/runtime support-এর অবস্থা version অনুযায়ী পরিবর্তিত হতে পারে।

তাই production project-এ ব্যবহার করার আগে target runtime/browser support যাচাই করা উচিত।

36.43 using / Explicit Resource Management

Modern ECMAScript-এর resource management feature হিসেবে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
using  
await using
```

ধারণা এসেছে। উদাহরণস্বরূপ কোনো resource:

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

```
File  
Connection  
Lock  
Stream
```

ব্যবহার শেষে automatically cleanup করার উদ্দেশ্যে। Concept:

JAVASCRIPT SOURCE

JAVASCRIPT

```
using resource = createResource();
```

Scope শেষ হলে resource disposal-এর mechanism কাজ করতে পারে।

💡 এটি advanced feature; সব environment-এ একইভাবে available কিনা target runtime অনুযায়ী verify করতে হবে।

Part 11 — Modern JavaScript Paradigms: Immutability, React & Node.js (36.44 – 36.49)

36.44 Modern JavaScript-এ Immutability

Modern frontend development-এ একটি বড় concept:

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

```
Mutation ✗  
Immutable Update ✓
```

পুরোনো:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const numbers = [1, 2, 3];  
  
numbers.reverse();
```

Original array পরিবর্তন। Modern:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const numbers = [1, 2, 3];  
  
const reversed = numbers.toReversed();
```

Original:

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

```
[1, 2, 3]
```

New:

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

```
[3, 2, 1]
```

36.45 Modern Object Handling

আগে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const hasName =
  Object.prototype.hasOwnProperty.call(
    user,
    "name"
  );
```

Modern:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const hasName =
  Object.hasOwn(user, "name");
```

Code অনেক cleaner!

36.46 Modern Promise Handling

ধরো তিনটি API:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const requests = [
  fetch("/users"),
  fetch("/products"),
  fetch("/orders")
];
```

সব সফল না হলেও result দরকার:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const results =
  await Promise.allSettled(requests);
```

একটি সফল হলেই result দরকার:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const result =
  await Promise.any(requests);
```

36.47 Modern Array Workflow

আগে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const copy = [...numbers];

copy.sort();
```

Modern:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const sorted = numbers.toSorted();
```

আগে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const copy = [...numbers];

copy.reverse();
```

Modern:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const reversed = numbers.toReversed();
```

আগে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const copy = [...numbers];

copy.splice(1, 1);
```

Modern:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const updated = numbers.toSpliced(1, 1);
```

36.48 Modern JavaScript + React

React development-এ তুমি অনেক জায়গায় এগুলো ব্যবহার করবে:

◆ Optional Chaining

★ MODERN JAVASCRIPT ARCHITECTURE FLOW

```
<p>{user?.profile?.name}</p>
```

◆ Nullish Coalescing

JAVASCRIPT SOURCE

JAVASCRIPT

```
const pageSize = settings?.pageSize ?? 10;
```

◆ Immutable Array

JAVASCRIPT SOURCE

JAVASCRIPT

```
const sortedUsers =
  users.toSorted(
    (a, b) => a.name.localeCompare(b.name)
  );
```

◆ findLast()

JAVASCRIPT SOURCE

JAVASCRIPT

```
const lastAdmin =
  users.findLast(user => user.role === "admin");
```

36.49 Modern JavaScript + Node.js

Node.js application-৯:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const result =
  await Promise.allSettled([
    fetchUsers(),
    fetchProducts(),
    fetchOrders()
  ]);
```

Error handling:

JAVASCRIPT SOURCE

JAVASCRIPT

```
try {

  await database.connect();

} catch (error) {

  throw new Error(
    "Database connection failed",
    { cause: error }
  );

}
```

Private class state:

JAVASCRIPT SOURCE

JAVASCRIPT

```
class Database {

  #connection;

  connect() {
    this.#connection = "connected";
  }

}
```

Part 12 — Browser Compatibility, Transpilation, Polyfills & ES2020+ Map (36.50 – 36.57)

36.50 Browser Compatibility

সব browser সব modern feature একই সময়ে support নাও করতে পারে। তাই production-এর আগে:

🌟 FULL-STACK MODERN JAVASCRIPT TOOLKIT

```
Browser support
Node.js version
Build tool
Babel
Transpiler
Polyfill
```

বিবেচনা করতে হয়।

36.51 Babel কী?

Babel হলো JavaScript compiler/transpiler ecosystem-এর একটি গুরুত্বপূর্ণ tool। ধরো তুমি modern syntax লিখলে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const result = user?.profile?.name;
```

পুরোনো environment-এর জন্য toolchain এটাকে compatible form-এ transform করতে পারে। তবে:

💡 সব নতুন API syntax transformation দিয়ে solve করা যায় না। কিছু API-এর জন্য polyfill বা runtime support দরকার।

36.52 Transpilation vs Polyfill

খুব গুরুত্বপূর্ণ distinction।

♦ Transpilation

Syntax পরিবর্তন করে। উদাহরণ conceptually:

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

Modern JS



Compiler



Older-compatible JS

♦ Polyfill

যে API নেই, তার implementation provide করে। যেমন:

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

Missing API



Polyfill



Available behavior

36.53 Feature Detection

কোনো feature support করে কিনা check করা যায়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
if ("at" in Array.prototype) {  
  console.log("Supported");  
}
```

অথবা:

JAVASCRIPT SOURCE

JAVASCRIPT

```
if (Promise.withResolvers) {  
  console.log("Supported");  
}
```

এটি runtime compatibility handling-এর জন্য usefull

36.54 Modern Syntax বনাম Modern API

এ দুটো আলাদা।

♦ Syntax

JAVASCRIPT SOURCE

JAVASCRIPT

```
user?.name
```

♦ API

JAVASCRIPT SOURCE

JAVASCRIPT

```
Promise.allSettled()
```

Syntax-এর জন্য transpiler কাজ করতে পারে। API-এর জন্য runtime/polyfill support প্রয়োজন হতে পারে।

36.55 ES2020+ Quick Version Map

Version	গুরুত্বপূর্ণ Features
ES2020	<code>?.</code> , <code>??</code> , <code>BigInt</code> , <code>Promise.allSettled()</code> , <code>Promise.any()</code>
ES2021	<code>replaceAll()</code> , logical assignment, numeric separators
ES2022	class fields, <code>private #</code> , static blocks, <code>.at()</code> , <code>Object.hasOwn()</code> , top-level <code>await</code> , <code>Error.cause</code>
ES2023	<code>findLast()</code> , <code>findLastIndex()</code> , <code>toSorted()</code> , <code>toReversed()</code> , <code>toSpliced()</code> , <code>with()</code>
ES2024	<code>Object.groupBy()</code> , <code>Map.groupBy()</code> , <code>Promise.withResolvers()</code> , well-formed string methods
ES2025	নতুন standardized language features; runtime support অনুযায়ী ব্যবহার

36.56 সবচেয়ে বেশি ব্যবহৃত Modern Features

বাস্তব project-এর জন্য priority:

🔥 Tier 1 — অবশ্যই জানবে

🌟 IMMUTABLE ARRAY METHODS PIPELINE

```
?.
??
async/await
Promise.allSettled()
Promise.any()
?? =
||=
&&=
Array.at()
toSorted()
toReversed()
toSpliced()
Object.hasOwn()
private #fields
```

🔥 Tier 2 — ভালোভাবে জানা উচিত

🌟 MODERN JAVASCRIPT ARCHITECTURE FLOW

```
Object.groupBy()
Map.groupBy()
Promise.withResolvers()
Error.cause
static fields
static blocks
top-level await
```

🔥 Tier 3 — Advanced

🌟 MODERN JAVASCRIPT ARCHITECTURE FLOW

```
using
await using
Temporal concepts
Unicode well-formed APIs
```

36.57 Modern JavaScript Interview Questions – ইন্টারভিউ প্রশ্নোত্তর

♦ Q1. || এবং ?? -এর difference কী?

JAVASCRIPT SOURCE

JAVASCRIPT

```
0 || 10
```

→ 10

JAVASCRIPT SOURCE

JAVASCRIPT

```
0 ?? 10
```

→ 0 কারণ ?? শুধু null এবং undefined handle করে।

♦ Q2. sort() এবং toSorted() difference?

JAVASCRIPT SOURCE

JAVASCRIPT

```
sort()
```

→ original array mutate করে।

JAVASCRIPT SOURCE

JAVASCRIPT

```
toSorted()
```

→ নতুন sorted array দেয়।

♦ Q3. Promise.all() এবং Promise.allSettled() ?

Promise.all():

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

একটি reject
→ পুরো Promise reject

Promise.allSettled():

✦ MODERN JAVASCRIPT ARCHITECTURE FLOW

সবগুলোর final result
→ fulfilled/rejected দুটোই

♦ Q4. Promise.any() কী করে?

প্রথম fulfilled Promise-এর result দেয়। সব reject হলে AggregateError দেয়।

♦ Q5. Optional chaining কেন ব্যবহার করি?

Nested property missing হলেও error avoid করতে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
user?.profile?.name
```

♦ Q6. Private class field কী?

JAVASCRIPT SOURCE

JAVASCRIPT

```
class User {  
  #password;  
}
```

#password class-এর বাইরে directly access করা যায় না।

♦ Q7. .at(-1) কী করে?

Array/String-এর শেষ element দেয়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
[10, 20, 30].at(-1)
```

→ 30

36.58 🔥 MUST KNOW — মডার্ন ECMAScript-এর ১০টি প্রধান স্তম্ভ

আধুনিক ফুলস্ট্যাক প্রজেক্টে (React & Node.js) প্রতিদিন কাজে লাগার মতো ১০টি সেরা উদ্ভাবন:

🌟 1. Null Safety & Operators

- Optional Chaining (?.):** নাল বা আনডিফাইন্ডে ক্র্যাশ ছাড়া এক্সেস
- Nullish Coalescing (??):** শুধুমাত্র null/undefined-এ ফলব্যাক (0 বা "" নিরাপদ)
- Logical Assignments:** ??=, &&=, ||= দিয়ে ক্লিন কনফিগ ইনিশিয়লাইজ
- Numeric Separators (`,`):** 1_000_000 লিখে বড় সংখ্যার পঠনযোগ্যতা বৃদ্ধি

🌟 2. Modern Classes & Async

- Private Fields (#):** টু ল্যাক্সয়েজ-লেভেল এনক্যাপসুলেশন (#balance)
- Static Blocks:** ক্লাসে কমপ্লেক্স স্ট্যাটিক মেম্বর ইনিশিয়লাইজেশন
- Top-Level await:** মডিউল লোডিংয়ের সময় অ্যাসিনক্রোনাস ডাটা ফেচ
- Promise.allSettled & any:** রেজলভ ও রিজেক্টেড প্রমিজের নির্ভরযোগ্য হ্যান্ডলিং

🌟 3. Immutable Data & Grouping

- toSorted / toSpliced:** মূল অ্যারে মিউটেট না করে নতুন অ্যারে রিটার্ন
- Array.prototype.at(-1):** নেগেটিভ ইনডেক্স দিয়ে লাস্ট উপাদান বের করা
- Object.groupBy():** ডাটাকে ক্যাটাগরি অনুযায়ী ডিক্টারেটিভ গ্রুপিং
- Error.cause:** এরর ট্র্যাজেক্টরি ও রুট কজ চেনিং

36.59 📄 Quick Cheat Sheet — আধুনিক সিনট্যাক্স ও মেথড সারসংক্ষেপ

Feature / Method	Introduction	Syntax / Code Snippet	Primary Benefit
Optional Chaining	ES2020	user?.address?.city	TypeError: Cannot read properties of undefined চিরতরে রোধ
Nullish Coalescing	ES2020	const port = customPort ?? 3000	ফলসি ভ্যালু (0, "") নষ্ট না করে শুধু null/undefined চেক
Promise.allSettled	ES2020	Promise.allSettled([p1, p2])	একটি ব্যর্থ হলেও সব প্রমিজের পূর্ণাঙ্গ ফলাফল পাওয়া যায়
String.replaceAll	ES2021	'a-b-c'.replaceAll('-', ' ')	গ্লোবাল রেজেক্স (/g) ছাড়াই সকল ম্যাচ প্রতিস্থাপন
Logical Assignment	ES2021	options.timeout ??= 5000	ভ্যালু নাল হলে তবেই শর্ট-সার্কিট অ্যাসাইনমেন্ট
Numeric Separator	ES2021	const salary = 1_500_000	কোডের ভেতরে বড় সংখ্যার পঠনযোগ্যতা বৃদ্ধি
Private Class Fields	ES2022	class Bank { #pin = 1234; }	ক্লাসের বাইরে থেকে ফিল্ড এক্সেস সম্পূর্ণ ব্লক
Array.prototype.at()	ES2022	arr.at(-1)	নেগেটিভ ইনডেক্স দিয়ে পেছনের উপাদান এক্সেস
Object.hasOwn()	ES2022	Object.hasOwn(obj, 'prop')	hasOwnProperty এর নিরাপদ ও সুরক্ষিত বিকল্প
Error Cause	ES2022	new Error('Fail', { cause: err })	অরিজিনাল এরর সংরক্ষণ করে রুট-কজ চেনিং
Array.toSorted()	ES2023	const sorted = arr.toSorted()	মূল অ্যারে অক্ষত রেখে নতুন সর্টেড অ্যারে তৈরি (React-safe)
Array.toSpliced()	ES2023	const removed = arr.toSpliced(0, 1)	ইমিউটেবল স্লাইসিং ও উপাদান অপসারণ
Array.with()	ES2023	const updated = arr.with(2, 99)	নির্দিষ্ট ইনডেক্সের মান পরিবর্তন করে ফ্রেশ কপি রিটার্ন
Array.findLast()	ES2023	arr.findLast(n => n > 10)	পেছনের দিক থেকে প্রথম ম্যাচিং উপাদান খুঁজে বের করা
Object.groupBy()	ES2024	Object.groupBy(users, u => u.role)	অ্যারের অবজেক্টগুলোকে শর্তানুযায়ী ক্যাটাগরিভাইজড অবজেক্টে রূপান্তর
Promise.withResolvers	ES2024	const { promise, resolve } = Promise.withResolvers()	প্রমিজ ও রিজলভ/রিজেক্ট ফাংশন সরাসরি ডিকনস্ট্রাক্ট

Part 13.1 — 8 Hands-On Practice Labs with Complete Production Solutions

Lab #1 [BEGINNER] — Falsy vs Nullish Evaluation Matrix (|| vs ??)

সমস্যা ও লক্ষ্য: নিচের তিনটি স্টেটমেন্টের এক্সিকিউশন আউটপুট কী হবে এবং 0 এর ক্ষেত্রে || এবং ?? কীভাবে ভিন্ন আচরণ করে তা ব্যাখ্যা করুন।

MODERN IMPLEMENTATION (LAB #1)

JAVASCRIPT

```
// Input Evaluation Code
console.log(0 || 50); // আউটপুট: 50
console.log(0 ?? 50); // আউটপুট: 0
console.log(null ?? 50); // আউটপুট: 50
```

💡 প্রোডাকশন অন্তর্দৃষ্টি: || (Logical OR) অপারেটর 0-কে falsy মনে করে ডান পাশের মান (50) নেয়। কিন্তু ?? (Nullish Coalescing) শুধুমাত্র null ও undefined-কে অগ্রাহ্য করে; 0 একটি সম্পূর্ণ বৈধ সংখ্যা হওয়ায় 0-ই রিটার্ন করে।

Lab #2 [BEGINNER] — Defensive Data Access with Optional Chaining (?.)

সমস্যা ও লক্ষ্য: nested অবজেক্ট থেকে ক্র্যাশ ছাড়া নিরাপদে ডাটা পড়তে Optional Chaining ও Nullish Coalescing ব্যবহার করে প্রোডাকশন ফাংশন লিখুন।

MODERN IMPLEMENTATION (LAB #2)

JAVASCRIPT

```
const userA = {
  profile: {
    address: { city: 'Dhaka', zip: 1205 }
  }
};

const userB = { profile: null };

// ❌ Risky: userB-তে TypeError ক্র্যাশ করবে
// const city = userB.profile.address.city;

// ✅ Modern Safe Navigation
function getCity(user) {
  return user?.profile?.address?.city ?? 'Unknown City';
}

console.log('User A City:', getCity(userA)); // Dhaka
console.log('User B City:', getCity(userB)); // Unknown City (No crash!)
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** ?. অপারেটর চেইনের কোনো ধাপে null বা undefined পেলে সাথে সাথে এক্সিকিউশন থামিয়ে undefined রিটার্ন করে, ফলে TypeError চিরতরে দূর হয়।

Lab #3 [BEGINNER] — Array End Indexing with Modern .at() Method

সমস্যা ও লক্ষ্য: Array-এর শেষ উপাদান বের করার জন্য ট্র্যাডিশনাল arr[arr.length - 1] এর বদলে আধুনিক .at(-1) মেথড প্রয়োগ করুন।

MODERN IMPLEMENTATION (LAB #3)

JAVASCRIPT

```
const numbers = [10, 20, 30, 40];

// Old Syntax
const lastOld = numbers[numbers.length - 1];

// Modern ES2022 Syntax
const lastModern = numbers.at(-1); // শেষ উপাদান: 40
const secondLast = numbers.at(-2); // শেষের আগের উপাদান: 30

console.log('Last Element:', lastModern);
console.log('Second Last:', secondLast);
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** .at() মেথডটি শুধু অ্যারেতেই নয়, স্ট্রিং (string.at(-1)) এবং টাইপড অ্যারেতেও সমভাবে কাজ করে যা কোডকে আরও সংক্ষিপ্ত ও আধুনিক করে তোলে।

Lab #4 [INTERMEDIATE] — True Class Encapsulation with Private Field (#balance)

সমস্যা ও লক্ষ্য: এমন একটি BankAccount ক্লাস তৈরি করুন যেখানে name পাবলিক, #balance প্রাইভেট এবং ভ্যালিডেশনসহ deposit() ও getBalance() মেথড থাকবে।

MODERN IMPLEMENTATION (LAB #4)

JAVASCRIPT

```
class BankAccount {
  name; // পাবলিক ফিল্ড
  #balance = 0; // ট্রু প্রাইভেট ফিল্ড (ক্লাসের বাইরে সম্পূর্ণ অদৃশ্য)

  constructor(accountHolder, initialDeposit = 0) {
    this.name = accountHolder;
    if (initialDeposit > 0) this.#balance = initialDeposit;
  }

  deposit(amount) {
    if (amount ≤ 0) {
      throw new Error('Deposit amount must be strictly positive');
    }
    this.#balance += amount;
    return this.#balance;
  }

  getBalance() {
    return this.#balance;
  }
}

const account = new BankAccount('Tanvir', 500);
account.deposit(250);
console.log(`${account.name} Balance: ${account.getBalance()}`); // $750

// ✗ SyntaxError: Private field '#balance' must be declared in an enclosing class
// console.log(account.#balance);
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** জাভাস্ক্রিপ্টে পূর্বে ব্যবহৃত আন্ডারস্কোর (_balance) কেবল একটি কনভেনশন ছিল যা বাইরে থেকে মডিফাই করা যেত। #balance হলো হার্ডওয়্যার/ল্যান্ডকুয়েজ-লেভেল প্রাইভেট মেম্বার।

Lab #5 [INTERMEDIATE] — Immutable Array Sorting with toSorted() (React Safe)

সমস্যা ও লক্ষ্য: মূল অ্যারে অপরিবর্তিত রেখে [10, 5, 30, 20, 15] অ্যারেকে descending অর্ডারে সাজানোর জন্য toSorted() মেথড ব্যবহার করুন।

MODERN IMPLEMENTATION (LAB #5)

JAVASCRIPT

```
const numbers = [10, 5, 30, 20, 15];

// ✗ Mutating sort(): মূল numbers অ্যারেকে স্থায়ীভাবে বদলে ফেলে
// numbers.sort((a, b) => b - a);

// ✅ Modern ES2023 Immutable toSorted()
const descending = numbers.toSorted((a, b) => b - a);

console.log('Original Array:', numbers); // [10, 5, 30, 20, 15] (অক্ষত!)
console.log('Descending Sorted:', descending); // [30, 20, 15, 10, 5]
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** React অ্যাপ্লিকেশনে মূল স্টেট মিউটেট করলে UI রি-রেন্ডারিং সংক্রান্ত গুরুতর বাগ তৈরি হয়। toSorted() কোনো ক্লোনিং ছাড়াই ইমিউটেবল নতুন অ্যারে প্রদান করে।

Lab #6 [INTERMEDIATE] — Fault-Tolerant Concurrent Requests with Promise.allSettled()

সমস্যা ও লক্ষ্য: তিনটি কনকারেন্ট এপিআই কলের মধ্যে কিছু প্রমিজ ফেইল করলেও যাতে পুরো প্রসেস ক্র্যাশ না করে প্রতিটি কলের স্টেট পরীক্ষা করুন।

MODERN IMPLEMENTATION (LAB #6)

JAVASCRIPT

```
const fetchA = Promise.resolve({ source: 'Payment Gateway', status: 'OK' });
const fetchB = Promise.reject(new Error('Auth Service Timeout (504)'));
const fetchC = Promise.resolve({ source: 'Notification Service', status: 'Delivered' });

async function loadSystemStatus() {
  const results = await Promise.allSettled([fetchA, fetchB, fetchC]);

  results.forEach((res, index) => {
    if (res.status === 'fulfilled') {
      console.log(`Service #${index + 1} [ACTIVE ✅]:`, res.value);
    } else {
      console.warn(`Service #${index + 1} [FAILED ❌]:`, res.reason.message);
    }
  });
}

loadSystemStatus();
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** Promise.all() একটি রিজেক্ট হলেই পুরো পাইপলাইন থামিয়ে দেয়। Promise.allSettled() সব প্রমিজ শেষ হওয়া পর্যন্ত অপেক্ষা করে fulfilled বা rejected স্ট্যাটাস রিপোর্ট করে।

Lab #7 [ADVANCED] — Declarative Data Categorization with Object.groupBy()

সমস্যা ও লক্ষ্য: ব্যবহারকারীর রোল (admin/user) অনুযায়ী ইউজার তালিকাকে অবজেক্ট গ্রুপিং করার জন্য আধুনিক ES2024 Object.groupBy() মেথড প্রয়োগ করুন।

MODERN IMPLEMENTATION (LAB #7)

JAVASCRIPT

```
const users = [
  { name: 'Tanvir', role: 'admin' },
  { name: 'Sakib', role: 'user' },
  { name: 'Rahim', role: 'admin' },
  { name: 'Karim', role: 'user' },
  { name: 'Ayesha', role: 'editor' }
];

// Modern ES2024 Declarative Grouping
const groupedByRole = Object.groupBy(users, (user) => user.role);

console.log(groupedByRole);

/*
আউটপুট:
{
  admin: [ { name: 'Tanvir', role: 'admin' }, { name: 'Rahim', role: 'admin' } ],
  user: [ { name: 'Sakib', role: 'user' }, { name: 'Karim', role: 'user' } ],
  editor: [ { name: 'Ayesha', role: 'editor' } ]
}
*/
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** পূর্বে আরের উপাদান গ্রুপ করতে জটিল Array.reduce() লজিক বা Lodash লাইব্রেরি ব্যবহার করতে হতো। Object.groupBy() এখন জাভাস্ক্রিপ্টের নেটিভ বিল্ট-ইন মেথড।

Lab #8 [ADVANCED] — Fastest Responsive API Race with Promise.any()

সমস্যা ও লক্ষ্য: একাধিক ব্যাকআপ এপিআইয়ের মধ্যে যেটি প্রথম সফল রেসপন্স দেবে সেটি গ্রহণ করুন এবং সবগুলো ফেইল করলে AggregateError হ্যান্ডেল করুন।

MODERN IMPLEMENTATION (LAB #8)

JAVASCRIPT

```
const mirrorCDN1 = new Promise((_, reject) => setTimeout(() => reject('CDN 1 Offline'), 100));
const mirrorCDN2 = new Promise((resolve) => setTimeout(() => resolve('CDN 2 High-Speed Data Payload'), 200));
const mirrorCDN3 = new Promise((resolve) => setTimeout(() => resolve('CDN 3 Data Payload'), 500));

async function fetchFastestAsset() {
  try {
    // যেটি প্রথম সফল হবে সেটিই ফলাফল দেবে (ব্যর্থতাগুলো ইগনোর করবে)
    const fastestResponse = await Promise.any([mirrorCDN1, mirrorCDN2, mirrorCDN3]);
    console.log('🔥 Connected via Fastest Mirror:', fastestResponse);
  } catch (error) {
    if (error instanceof AggregateError) {
      console.error('All mirror servers failed:', error.errors);
    }
  }
}

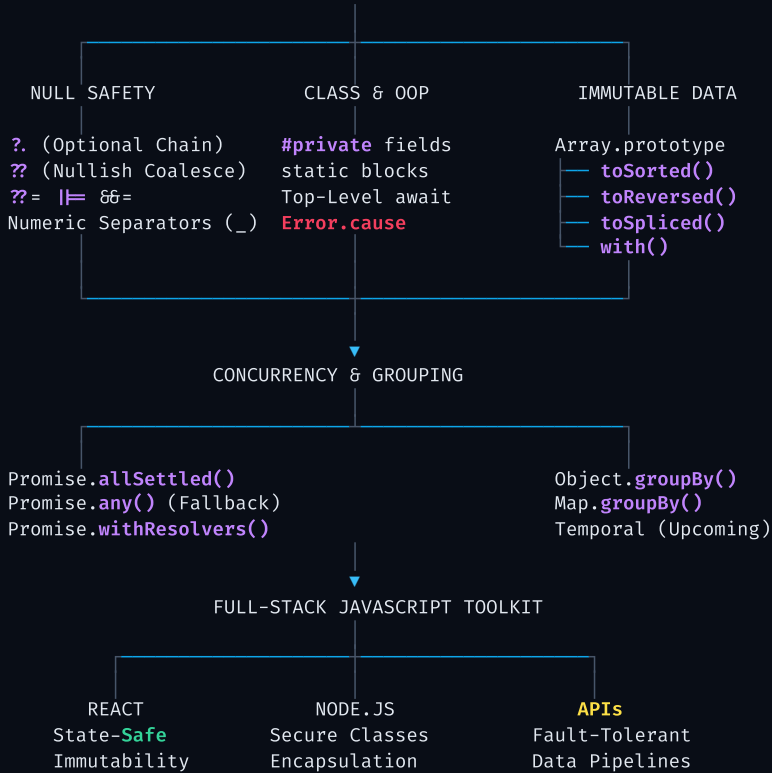
fetchFastestAsset();
```

💡 **প্রোডাকশন অন্তর্দৃষ্টি:** Promise.race() প্রথম প্রমিজটি ফেইল করলে নিজেও ফেইল করে। কিন্তু Promise.any() প্রথম সফল প্রমিজটি গ্রহণ করে এবং সবগুলো ফেইল করলেই কেবল AggregateError ত্রুটি করে।

পুরো Chapter 36-এর আধুনিক ফিচার ও ফুলস্ট্যাক অ্যাপ্লিকেশন একসাথে:

🌟 MODERN ECMAScript ARCHITECTURE MAP

MODERN ECMAScript ECOSYSTEM



👤 এক লাইনে Chapter 36: Modern ECMAScript (ES2020+) হলো নাল-সেফটি, ইমিউটেবল ডাটা ম্যানিপুলেশন, ক্লাস এনক্যাপসুলেশন ও কনকারেন্ট প্রমিজ পরিচালনার এমন এক সমন্বিত রূপ যা রিঅ্যাক্টি ফ্রন্টএন্ড ও নোড ব্যাকএন্ড কোডকে করে তোলে আরও সংক্ষিপ্ত, ক্র্যাশ-প্রুফ ও পারফরম্যান্ট।

🔗 Curriculum Progression: Chapter 35 → 36 → 37

🌟 ROADMAP CONTINUITY PIPELINE

Chapter 35: Testing JavaScript



Unit Testing, Integration Testing, Assertions, Jest/Vitest, Mocking & CI/CD



Chapter 36: Modern ECMAScript / ES2020+ Features (THIS CHAPTER)



Optional Chaining, Nullish Coalescing, Top-level Await, toSorted, Object.groupBy



Chapter 37: JavaScript Design Patterns



Singleton, Factory, Observer, Module, Prototype, Proxy & MVC/MVVM Architectural Patterns